

DevOpsAgent AI

Autonomous SRE & Infrastructure Self-Healing Engine (Taskmaster Track)

Live Cloud Run URL:	https://devops-agent-service-785389711688.us-central1.run.app/
GitHub Repository:	https://github.com/creationsofh/devops-agent
GCP Project ID:	project-d1771649-7eea-41e2-939
Tech Stack:	Python, FastAPI, Streamlit, Google Cloud Run, Google GenAI SDK, GCP APIs

1. EXECUTIVE SUMMARY & DUAL-MODE OVERVIEW

What is DevOpsAgent AI?

DevOpsAgent AI is an autonomous, production-grade Site Reliability Engineering (SRE) assistant designed to convert raw infrastructure errors, telemetry logs, and stack traces into immediate operational remediation. Built with a **Dual Execution Mode Architecture**: when Google Cloud billing credits are active, it issues live client calls to Cloud Run v2, Stackdriver Logging, and IAM APIs. If credits expire, it automatically transitions to simulated fallback execution without requiring code updates or breaking UI flows.

2. TARGETED AUDIENCE & PROBLEM STATEMENT

For Whom is this Built?

- **DevOps Engineers & SREs:** Eliminates manual triage for container crashes, permission blockages, and log inspection.
- **Cloud & Full-Stack Developers:** Resolves model deprecation errors and infrastructure bottlenecks without leaving the chat interface.
- **On-Call Incident Response Teams:** Accelerates mean-time-to-resolution (MTTR) during active cloud outages.

Problem Solutions Matrix:

INDUSTRY PAIN POINT	TRADITIONAL IMPACT	DEVOPSAGENT AI SOLUTION
Cascading Model Deprecations	Codebases break silently when LLM API endpoints are deprecated.	Codebase Auto-Healing (<code>update_model_config</code>): Detects version 404s and automatically patches local source files.
Unobserved Microservices	Engineers spend hours filtering raw Stackdriver logs across services.	Live Telemetry Triage (<code>query_cloud_logging</code>): Queries Stackdriver APIs directly to isolate error stack traces.
Deadlocked Containers	High MTTR due to manual log inspection and gcloud restarts.	Microservice Lifecycle (<code>restart_cloud_run_service</code>): Triggers container revision updates on Cloud Run instantly.
Deployment Access Blocks	Developers wait for IAM admins to deploy urgent hotfixes.	JIT Access Provisioning (<code>grant_temporary_iam_role</code>): Grants

Industry Pain Point	Traditional Impact	DevOpsAgent AI Solution
		temporary scoped IAM roles upon policy evaluation.

3. SYSTEM ARCHITECTURE & DUAL-MODE EXECUTION

Hybrid Decoupled Architecture

The system runs as a containerized dual-process microservice on Google Cloud Run with warm minimum instances (`--min-instances 1`) to eliminate cold-start latency. Streamlit manages user interactions on port 8080 while FastAPI processes agent orchestrations on port 8000.

Execution Flow:

- Telemetry Intake:** User inputs incident logs via text or file attachment (`.log/.txt` up to 10MB).
- FastAPI Orchestration:** Payload is processed by the backend engine via the Google GenAI SDK using candidate model fallbacks.
- Dynamic Tool Invocation:** If actionable telemetry is identified, Gemini emits structured function calls matching the backend tool map: `query_cloud_logging` `restart_cloud_run_service` `grant_temporary_iam_role` `update_model_config`
- Dual-Mode Execution & Audit:** Evaluates live GCP API availability. Executes live Cloud Run/Logging operations when credits are active, or falls back to simulated cards if restricted. Formatted JSON results render in custom Streamlit audit cards.

4. KEY DIFFERENTIATORS

- Action-Oriented Architecture:** Bypasses pure conversational text output to execute operational remediation directly.
- Dual-Mode API Resilience:** Features automatic 429 rate-limit handling and dynamic fallbacks between real GCP client APIs and simulated mock modes.
- Integrated Visual Audit Cards:** Renders custom HTML execution cards detailing function names, arguments, and execution statuses.

5. AUTONOMOUS CAPABILITY SUITE

Capability	Tool Function	Real GCP Mode	Free-Tier Fallback Mode
Live Telemetry Triage	<code>query_cloud_logging</code>	Queries Google Stackdriver API for real stack traces	Returns structured mock incident traces
Cloud Run Lifecycle	<code>restart_cloud_run_service</code>	Executes real revision updates via <code>run_v2.ServicesClient</code>	Simulates container restart logs
JIT Access Provisioning	<code>grant_temporary_iam_role</code>	Adds IAM policy bindings via <code>gcloud</code> CLI	Simulates role elevation policy card
Codebase Self-Repair	<code>update_model_config</code>	Patches local source code files dynamically	Patches local source code files dynamically

6. COMPREHENSIVE DEVPOST USER MANUAL

A. Testing Live Cloud Logging Inspection

Paste the following prompt into the chat interface:

```
Fetch recent ERROR stack traces for Cloud Run service devops-agent-service.
```

Expected Result: Triggers `query_cloud_logging` and renders log traces directly from Google Cloud Logging.

B. Testing Cloud Run Microservice Restart

Paste the following stack trace into the chat interface:

```
ALERT: order-processing-service in us-central1 is throwing HTTP 500 errors due to process deadlock.  
Please restart the microservice.
```

Expected Result: Triggers `restart_cloud_run_service` targeting `order-processing-service` in `us-central1`.

C. Testing Infrastructure Self-Healing

Paste the following model deprecation trace into the chat interface:

```
Execution Error: 404 NOT_FOUND. Model models/gemini-2.5-flash is deprecated. Please update code to use  
models/gemini-3.6-flash.
```

Expected Result: Triggers `update_model_config` and patches backend/frontend source files.

D. Testing Advisory Queries

```
What are the best practices for setting up Docker multi-stage builds for FastAPI apps?
```

Expected Result: The agent recognizes no infrastructure modification is needed and provides structured advisory guidance.